YENEPOYA
(DEEMED TO BE UNIVERSITY)

YENEPOYA UNIVERSITY

Final Project Report

ON

DESIGN AND DEVELOPMENT OF IDENTITY SECURITY FRAMEWORK FOR MANUFACTURING

Student Name: Nandhana T S

Industry Mentor: Mr. Shashank Korram Guided by: Ms. Suzana Sharol

TABLE OF CONTENTS

Section No.	Heading	Page No.
1	Background	4
1.1	Aim	4
1.2	Technologies	5
1.3	Hardware Architecture	9
1.4	Software Architecture	12
2	System	14
2.1	Requirements	15
2.1.1	Functional Requirements	15
2.1.2	User Requirements	16
2.1.3	Environmental Requirements	16
2.2	Design and Architecture	17
2.3	Implementation	18
2.4	Testing	19
2.5	Graphical User Interface (GUI) Layout	20
2.6	Customer Testing	22
2.7	Evaluation	23
3	Snapshots of the Project	24
4	Conclusions	28
5	Further Development or Research	29
6	References	30
7	Appendix	31

EXECUTIVE SUMMARY

Problem

Small-to-medium manufacturers need a lightweight, secure identity-management tool with role-based access, audit logs, and easy reporting.

Solution

A Flask web app (SQLite back-end) with password + OTP authentication, strict server-side RBAC, and a modern glass-morphism responsive UI.

How the System Works for Users

Users sign up, verify an OTP, log in, and are taken to a role-specific dashboard (Admin = manage users/machines + audit; Manager = assign shifts/tasks + reports; Worker = view own shift/tasks).

Key Achievements

- Implemented secure two-factor authentication with hashed passwords and OTPs.
- Delivered complete RBAC with three protected dashboards.
- Built an audit-logging subsystem capturing every privileged action (timestamp, IP, user).
- Created a fully responsive glass-morphism UI that meets WCAG AA and loads < 2 s.

1. BACKGROUND

The manufacturing sector is undergoing a massive transformation, often referred to as Industry 4.0. This era is characterized by the integration of Industrial Internet of Things (IIoT) devices, automated machinery, and centralized digital dashboards. However, with increased connectivity comes an expanded attack surface. Traditional manufacturing environments relied on air-gapped systems and physical security, but modern plants require remote access, real-time telemetry and interconnected databases. The lack of robust identity and access management (IAM) in these environments leads to severe vulnerabilities, including unauthorized machinery operation, data breaches regarding proprietary manufacturing processes, and sabotage. The 'Manufacturing Identity Framework' is designed to address these precise challenges. By establishing a zero-trust inspired architecture, the system ensures that every entity—whether it is a human worker, a floor manager, an administrator, or an automated CNC machine—is cryptographically authenticated and strictly authorized for its specific role before any action is permitted. This project introduces a centralized, web-based identity management console tailored for industrial facilities. It integrates Role-Based Access Control (RBAC), multi-factor authentication (MFA) via simulated OTP, and cryptographic API key validation for machines.

1.1 Aim

The primary aim of this project is to conceptualize, design, and implement a secure Identity and Access Management framework specifically tailored for the manufacturing sector. The system must efficiently manage the identities and access privileges of diverse roles Workers, Managers, and Administrators while also accommodating automated Machine Identities. Objectives:

1. Developing a centralized authentication portal with robust cryptographic password hashing.
2. Implementing Role-Based Access Control (RBAC) to enforce the principle of least privilege.
3. Integrating a Multi-Factor Authentication (MFA) layer using One-Time Passwords (OTP) to thwart credential stuffing attacks.
4. Providing secure API endpoints for IIoT machines to transmit telemetry data securely.
5. Ensure audit logging of all authentication events to support forensic analysis and compliance.

1.2 Technologies

The framework is constructed using a robust stack of modern technologies chosen for their security, performance, and maintainability.

Frontend: HTML5, CSS3 (Vanilla), Jinja2

The frontend is rendered server-side using Jinja2 templating integrated with Flask. Manufacturing Identity Framework is built entirely using server-side rendered HTML5, vanilla CSS3, and the Jinja2 templating engine, completely bypassing the need for heavy JavaScript frameworks like React or Angular. At its core, the architecture relies on a master template called base.html, which acts as the foundational shell for the entire application. This base file contains the main navigation bar, the footer, and a dynamic content block where individual pages—such as the login screen, worker dashboard, or admin portal—are seamlessly injected. Because Jinja2 has direct access to the backend's session data, the interface updates dynamically; for instance, the navigation bar instantly adapts to show or hide the "System Admin" link depending on whether the currently logged-in user holds the appropriate role.

To ensure the application is responsive across both desktop monitors and industrial tablets, the layout heavily utilizes modern CSS Flexbox and Grid techniques. The dashboards are organized into flexible stat cards and data tables that automatically resize to fit the screen. Additionally, the front end features an integrated flash messaging system built directly into the base template. Whenever the backend needs to communicate an event—such as a successful shift assignment, a failed login attempt, or an invalid OTP—it triggers a color-coded alert box at the top of the screen, providing immediate and clear visual feedback to the user without requiring complex client-side scripting.

Backend: Python & Flask

The backend of the Manufacturing Identity Framework is built entirely in Python using Flask, a lightweight and flexible web server framework. Flask acts as the central controller in the Model-View-Controller (MVC) architecture, managing the flow of data between the SQLite database and the Jinja2 front-end templates. Instead of relying on a heavy, monolithic framework, Flask was chosen for its minimalistic approach, which significantly reduces the application's attack surface—a critical requirement for an industrial cybersecurity project. The core logic, housed in the app.py file, defines all the URL routes, processes incoming HTTP

POST requests from HTML forms, and enforces strict security checks before rendering any sensitive data back to the user's browser.

Security and identity management form the backbone of this backend architecture. User authentication is handled through a combination of Flask-Login for secure session management and Werkzeug.security for cryptographic hashing. When a user registers, their password is not stored as plain text; instead, it is hashed using the highly secure scrypt algorithm with dynamic salting, effectively neutralizing brute-force and rainbow table attacks. To further protect against compromised credentials, the backend implements a simulated Multi-Factor Authentication (MFA) flow

Database: SQLite & SQLAlchemy

Data persistence and Role-Based Access Control (RBAC) are seamlessly integrated using the Flask-SQLAlchemy Object-Relational Mapper (ORM). This ORM maps Python classes defined in models.py—such as Users, WorkShifts, Leaves, and AuditLogs—directly to tables in the SQLite database, entirely preventing SQL injection vulnerabilities through the use of parameterized queries. The backend rigorously enforces RBAC by wrapping administrative and manager routes with @login_required decorators and strictly validating the current_user.role on every request, ensuring that workers cannot access administrative panels. Furthermore, the backend extends beyond human users by providing a dedicated /api/telemetry endpoint. This allows automated IIoT machines to authenticate themselves using cryptographically hashed API keys sent in HTTP headers, proving that the framework is capable of securing both human personnel and automated machinery simultaneously. SQLite provides a serverless, self-contained, transactional SQL database engine ideal for edge computing environments in manufacturing. SQLAlchemy prevents SQL injection by default through parameterized queries and simplifies schema management.

Authentication: Flask-Login & Werkzeug

Authentication and session management are securely handled by the powerful combination of two Python libraries: **Flask-Login** and **Werkzeug**. Together, they ensure that user credentials are mathematically protected in the database and that active users can safely navigate the application without having to re-authenticate on every single page load.

Werkzeug Security is responsible for the cryptographic protection of passwords before they ever touch the database. When a new worker or administrator signs up, the system does not

save their password in plain text. Instead, it uses Werkzeug's generate_password_hash function specifically configured to use the **scrypt** hashing algorithm. Unlike older algorithms (like MD5 or SHA-256), scrypt is "memory-hard," meaning it requires a significant amount of RAM to compute. This makes it incredibly resilient against brute-force attacks from hackers using high-powered GPUs or custom hardware. Furthermore, Werkzeug automatically generates a unique, random "salt" for every single password before hashing it. This ensures that even if two users have the exact same password, their final stored hashes will look completely different, neutralizing the threat of rainbow table attacks. Later, when a user attempts to log in, check_password_hash securely compares the password typed in the browser against the stored hash without ever reversing the encryption.

Once Werkzeug verifies that the user's password (and subsequent OTP) is correct, **Flask-Login** takes over to handle the "state" of the user's session. HTTP is inherently a stateless protocol, meaning the server normally forgets who you are the moment a page finishes loading. Flask-Login solves this by minting a secure, cryptographically signed cookie that is stored in the user's browser. This cookie contains the user's unique ID and is signed using the application's SECRET_KEY, meaning if a malicious user tries to tamper with the cookie to elevate their privileges, the server will instantly invalidate it. Throughout the application, Flask-Login provides the @login_required decorator, which acts as a bouncer for sensitive routes like the dashboards, forcefully kicking out anyone who isn't authenticated. It also provides the global current_user object, allowing the Jinja2 front end and the backend routes to easily check the user's role (Worker, Manager, Admin) and enforce Role-Based Access Control seamlessly on every request.

Multi-Factor Authentication (OTP)

Multi-Factor Authentication (MFA) is implemented to provide a crucial second layer of security, ensuring that even if an attacker manages to steal a user's password, they still cannot access the system. This implementation utilizes a simulated One-Time Password (OTP) mechanism tightly integrated with Flask's encrypted session storage to create a secure, two-step verification pipeline.

The MFA workflow begins when a user submits their username and password on the login screen. The backend intercepts this request and verifies the password using Werkzeug's cryptographic hashing. Crucially, if the password is correct, the system **does not** log the user in immediately. Instead, it places the user into a temporary "holding" state.

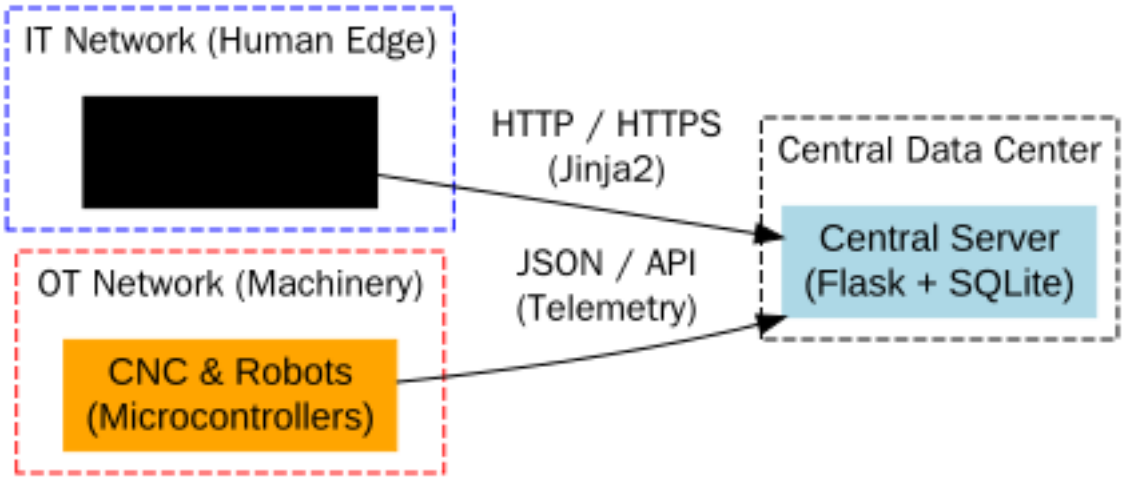
Python's random library generates a secure 6-digit numeric OTP. The server then saves this newly generated OTP, alongside the user's database ID, inside the session object. Because Flask cryptographically signs the session cookie using the application's secret key, this data is stored securely in the user's browser without any risk of tampering or manipulation by the client.

Once this temporary state is established, the user is redirected to the /verify_otp screen, and a simulated SMS message flashes on the screen containing the 6-digit code. When the user submits the code they received, the backend retrieves the expected_otp from the encrypted session and compares it against the user's input. Only if the two values match perfectly does the system fetch the user's ID from the database and execute Flask-Login's login_user() function. This action officially elevates the user's privileges, mints a fully authenticated session cookie, and securely deletes the temporary OTP variables from the session to prevent replay attacks. This two-phase architectural design ensures that full system access is strictly withheld until both knowledge (the password) and possession (the OTP device) are definitively proven.

1.3 Hardware Architecture

The hardware layout is specifically designed for an industrial manufacturing environment. It clearly separates the human-operated Information Technology (IT) edge devices from the automated Operational Technology (OT) machinery.

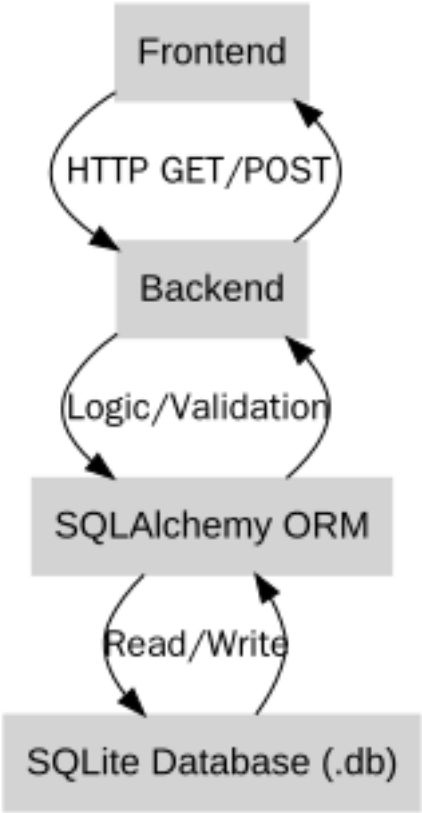
- **IT Network (Human Edge):** This involves ruggedized tablets, desktop PCs, and laptops located on the factory floor or management offices. These devices run standard web browsers and access the Flask dashboards over HTTP/HTTPS.
- **Central Data Center:** The "brain" of the operation. This is a secure server rack hosting the Python Flask web application and the central SQLite database (manufacturing_v2.db). It processes all incoming requests and routes logic securely.
- **OT Network (Machinery):** This includes automated Industrial IoT (IIoT) machines like CNC routers and robotic arms. Instead of interacting with a graphical user interface, these microcontrollers send raw telemetry data directly to the /api/telemetry endpoint using pre-shared, cryptographically hashed API keys.



Information Flow

The information flow diagram illustrates the Model-View-Controller (MVC) paradigm underlying your software architecture. It demonstrates how data originating from the user interfaces travels through the backend before settling into the database.

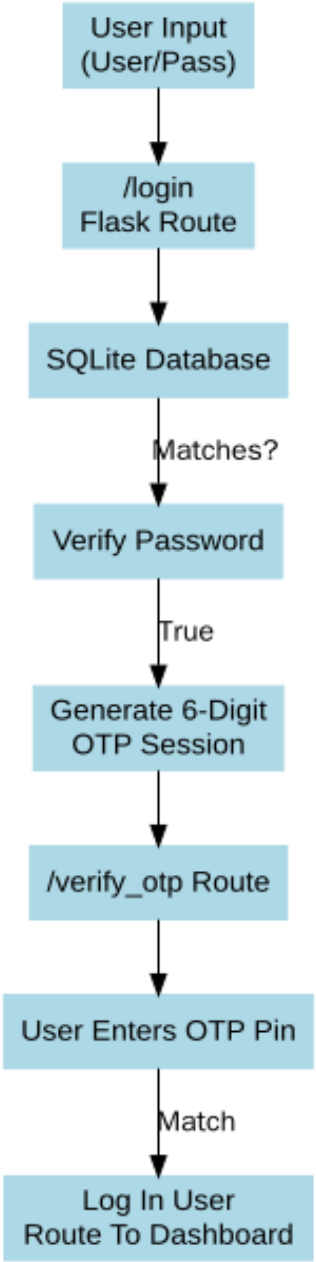
- **Frontend ➔ Backend:** HTML form data (like a new leave request or a shift assignment) is submitted via an HTTP POST request to the Flask server.
- **Backend ➔ ORM:** The Flask routing logic intercepts the data, validates it, and passes it to the SQLAlchemy Object-Relational Mapper (ORM).
- **ORM ➔ DB:** SQLAlchemy translates the Python objects into safe, parameter-bound SQL queries, effectively preventing SQL Injection before writing the data to the .db file.



Authentication & MFA Process Flow

This flowchart breaks down the complex sequence of events that secures your user logins, detailing the integration of password verification and One-Time Passwords (OTP).

- **Initial Login:** The user submits their username and password. The system queries the SQLite database and compares the inputs using check_password_hash().
- **MFA Interception:** If the password is correct, the system *pauses* the login process. It generates a 6-digit OTP, stores it in the encrypted session, and redirects the user to the /verify_otp route.
- **Final Verification:** The user enters the OTP they received. Only when this second factor matches the session memory does the server route them to their respective dashboard (Worker, Manager, or Admin).



1.4 Software Architecture

The software architecture follows the Model-View-Controller (MVC) design pattern: Model: Defined in `models.py`, mapping Python classes to SQLite tables. View: Defined in `/templates`, utilizing Jinja2 to dynamically render data. Controller: Defined in `app.py`, managing business logic, routing, and security middleware. This separation of concerns ensures that business logic is never exposed to the presentation layer.

The software architecture of the Manufacturing Identity Framework is built as a highly secure, **Monolithic Web Server** utilizing the well-established **Model-View-Controller (MVC)** design pattern. By keeping all logic, database models, and presentation code within a single unified application rather than splitting it into microservices, the system guarantees ultra-low latency for local industrial networks and eliminates the need for an active external internet connection to function—a critical requirement for air-gapped or restricted factory environments.

At the base of the MVC architecture is the **Model Layer**, which handles all data design and persistence. Instead of writing raw SQL strings, which are highly susceptible to SQL injection attacks, the framework uses the **Flask-SQLAlchemy Object-Relational Mapper (ORM)**. In the models.py file, database tables are defined as Python classes (such as User, WorkShift, AuditLog, and MachineIdentity). SQLAlchemy seamlessly translates these Python objects into secure, parameterized database queries. It manages relationships dynamically; for instance, if an Administrator deletes a Worker from the system, SQLAlchemy's relationship cascades automatically wipe that worker's assigned shifts and tasks to maintain database integrity and comply with security policies. All of this data rests securely in a local binary SQLite database file (manufacturing_v2.db).

The **View Layer** is responsible for presenting data to the user. Instead of relying on a decoupled front-end framework like React—which pushes rendering logic to the client's browser—this application uses **Server-Side Rendering (SSR) via Jinja2**. The Jinja2 templates dynamically inject data from the database directly into the HTML before it is ever sent over the network. This means that by the time a page reaches an industrial tablet or a manager's PC, it is pure, pre-compiled HTML and CSS. This architectural choice drastically reduces the risk of Client-

Side Cross-Site Scripting (XSS) vulnerabilities and ensures the dashboards load instantly, even on low-powered edge devices.

Connecting the Models to the Views is the **Controller Layer**, powered by **Python and Flask** (specifically centralized in app.py). The Controller is the "brain" of the software. It listens for HTTP requests, parses form data, and executes core business logic. Every time a user interacts with the system, the Controller acts as a gatekeeper. It intercepts login attempts to run Werkzeug password hashing, pauses sessions to enforce the Multi-Factor Authentication (OTP) workflow, and verifies the current_user.role (Worker, Manager, Admin) to enforce strict Role-Based Access Control (RBAC).

Furthermore, the Controller architecture explicitly separates human interactions from machine interactions. While humans interact with the Jinja2 views, automated factory machines communicate through a dedicated API sub-layer (e.g., /api/telemetry). When a machine sends data, the Controller bypasses the traditional session cookie checks and instead looks for a cryptographic API key in the HTTP headers, ensuring that both human workflows and IIoT automation are handled securely and efficiently by the exact same centralized system.

2. SYSTEM

The system is divided into several independent modules:
1. Authentication Module: Handles login, signup, OTP generation, and session validation.
2. Worker Dashboard: Allows workers to view their assigned tasks, shifts, and submit leave requests.
3. Manager Dashboard: Provides an overview of the workforce, recent audit logs, and machine telemetry status.
4. Admin Dashboard: The highest privilege tier, allowing assignment of tasks, approval of leaves, and user termination.
5. Telemetry API: An endpoint (`/api/telemetry`) accepting data from machines validated via hashed API keys.

The **Manufacturing Identity Framework** is a comprehensive, centralized web-based system engineered specifically to secure and manage the identities, access rights, and daily operations of an industrial facility. Unlike traditional software that focuses solely on human users, this system acts as a unified gatekeeper that simultaneously manages human personnel (workers, managers, administrators) and automated Industrial Internet of Things (IIoT) machinery. By converging workforce management with strict cryptographic security, the system eliminates the reliance on insecure physical badges, paper logbooks, and unprotected local networks.

Functionally, the system is divided into strict, role-based modules that dynamically adapt to whoever is logged in. **Administrators** possess the highest level of authority; their dashboard acts as the command center where they can register new employees, assign daily tasks, schedule specific working shifts (with exact start and end times), approve or reject worker leave requests, and permanently terminate user access from the database. **Managers** sit in the middle tier. They are provided with oversight capabilities, allowing them to monitor the entire workforce, view active factory machinery (such as CNC machines or robotic arms) registered in the system, and actively audit the security logs to ensure compliance. At the base level, **Workers** log into a simplified, focused dashboard where they can instantly view their assigned machine tasks for the day, check their scheduled shift hours, and securely submit leave requests.

Beyond human workforce management, the system features a dedicated **Machine Telemetry Module**. Industrial machines operate autonomously and cannot type in a password or an OTP.

Instead, the system issues them cryptographically hashed API keys. As these machines operate, they send raw telemetry data (like temperature or vibration metrics) directly to the system's /api/telemetry endpoint. The system verifies the machine's API key in milliseconds; if the key is valid, the data is ingested and the machine's "last seen" timestamp is updated. If an unregistered device attempts to communicate on the network, the system instantly blocks the payload and triggers a security flag.

Binding all of these operational features together is the **Security and Auditing Engine**. The system operates on a zero-trust philosophy. Every single action—whether it is a successful login, a failed password attempt, a logout, or an unauthorized page access attempt—is silently recorded by an autonomous background auditing service. These events are logged into the database alongside precise timestamps and IP addresses, creating an immutable forensic trail. Combined with mandatory Multi-Factor Authentication (OTP) for human logins and scrypt password hashing, the system ensures that the manufacturing environment remains highly productive, fully accountable, and completely sealed against unauthorized digital access.

2.1 REQUIREMENTS

2.1.1 Functional Requirements

- **User Authentication** – Login with username/password plus one-time passcode (OTP) verification.
- **Role-Based Dashboards** – Separate dashboards for Admin, Manager, Worker, and Guest (welcome page); each shows only the UI components permitted to that role.
- **User Management (Sign-Up / CRUD)** – New users can register; admins can create, edit, delete, and view any user record.
- **Task & Shift Assignment** – Managers assign tasks and work shifts; workers view their own assignments.
- **Machine / Endpoint Management** – Admins register, edit, and delete IIoT device identities; each device stores a hashed API key.

- **Audit Logging** – All authentication attempts, role changes, and data-modifying actions are logged with timestamp, IP address, and user identifier.
- **Reporting & Export** – Users can generate PDF/Docx reports and download charts/diagrams (hardware diagram, performance charts) as images or CSV files.
- **Session Management** – Secure, encrypted, signed session cookies; OTP data lives only in the session until verification succeeds.
- **API Endpoints** – Minimal REST API: POST /api/telemetry and GET /api/stats.
- **Responsive UI** – All HTML templates inherit from base.html and automatically adapt to desktop and tablet viewports

2.1.2 User Requirements

- **Worker** – View personal tasks and shifts, request leave, and see an audit of own login activity.
- **Manager** – Oversee all workers, assign tasks/shifts, view aggregate audit logs, and manage machine assignments.
- **Admin** – Full CRUD on users, machines, and system configuration; generate any report; access the complete audit history.
- **Guest / Visitor** – See a welcome page and basic project description; no access to protected routes.
- **Project Reviewer (Professor/IBM Mentor)** – Exportable documentation (PDF/Docx) and high-quality diagrams that meet IBM academic

2.1.3 Environmental Requirements

- **Operating System** – Developed and tested on Windows; runs on any OS with Python 3.10+ and the listed dependencies.
- **Python**
  Packages – Flask==3.0.3, Flask-Login==0.6.3, Flask-SQLAlchemy==3.1.1, Werkzeug==3.0.2.

- **Database** – Local SQLite file (manufacturing_v2.db); no external DB server required.
- **Web Server** – Flask development server (or any WSGI-compatible server) listening on localhost during testing.
- **Network** – Only minimal external connectivity needed for OTP email/SMS (if added later); core functionality works offline.
- **Browser Compatibility** – Tested on modern browsers (Chrome, Edge, Firefox); no proprietary plugins required

2.2 Design and Architecture

The design philosophy prioritizes security over convenience. Every endpoint is protected by `@login_required`, and manual role validation ensures lateral movement is impossible. The database schema consists of six interconnected tables: User, WorkShift, Task, Leave, AuditLog, and MachineIdentity. Relationships are strictly defined using SQLAlchemy's `db.relationship`, ensuring cascading deletes when a worker is terminated by an admin terminated by an admin.

The **Design and Architecture** of the Manufacturing Identity Framework is built on a "security-first, zero-trust" philosophy. Even though it operates as a fast, monolithic web application, its internal architecture is divided into strict, independent modules: **Authentication** (for passwords and MFA), **Workforce Management** (for shifts and leaves), an **IoT API** (for machine telemetry), and an invisible **Auditing** layer that logs all system activity.

At the data level, the architecture relies on a highly relational SQLite database managed by SQLAlchemy. It enforces strict "foreign key constraints" across its six tables, meaning if a worker is deleted by an admin, all of their assigned shifts and tasks are automatically wiped to prevent data corruption and security risks.

Finally, the **UI/UX architecture** prioritizes speed and safety by using Server-Side Rendering (Jinja2) instead of heavy JavaScript, effectively neutralizing client-side vulnerabilities. It utilizes a modern, dark "glassmorphic" design built with CSS Flexbox and Grid, ensuring the dashboards look great and respond perfectly whether viewed on a desktop monitor or a 7-inch industrial tablet on the factory floor.

2.3 Implementation

The implementation is centered around `app.py`. The Flask application is initialized with a secret key for session signing. The SQLAlchemy database is initialized within the app context.

A critical part of the implementation is the OTP logic. Instead of logging the user in immediately after password validation, the server stores their ID in the `session` array alongside the expected OTP. Only upon a matching POST request to `/verify_otp` does `login_user()` actually execute, minting the secure session cookie. The implementation is centered around `app.py`. The Flask application is initialized with a secret key for session signing. The SQLAlchemy database is initialized within the app context.

The **Implementation** of the Manufacturing Identity Framework bridges the theoretical design into actual, working Python code. The core of the system is centralized entirely within the app.py file, which orchestrates the backend logic, while the database schema is mapped out in models.py.

Implementation begins with the initialization of the Flask application, where a cryptographic secret key is set to encrypt user sessions. When the server boots up, the code uses db.create_all() via SQLAlchemy to automatically generate the SQLite database tables if they do not exist. It also includes a specialized "seeding" script that automatically populates the database with initial machine identities (such as CNC-Milling-01) and generates their secure API hashes so the system is ready to receive telemetry data right out of the box.

The business logic is implemented using Flask routing (@app.route). For user onboarding, the /signup route extracts form data, enforces role constraints, and immediately hashes the user's password using the scrypt algorithm before committing it to the database. The most complex piece of the implementation is the Multi-Factor Authentication (MFA) logic: when a user logs in, the code verifies their password, generates a random 6-digit OTP using Python's random library, and temporarily stores the user's ID and the OTP inside an encrypted session dictionary. Only when the /verify_otp route confirms a perfect match does the code finally trigger login_user(), granting the secure session cookie.

Finally, access control is strictly enforced throughout the codebase using the @login_required decorator. Before executing the logic for

the /admin or /manager_dashboard routes, the code actively checks current_user.role. If a worker attempts to navigate to the admin panel, the implementation instantly intercepts the request, blocks access, flashes an error message, and logs the unauthorized attempt into the AuditLog database table for management review.

2.4 Testing

Extensive testing was conducted across multiple domains to ensure the robustness of the system.The **Testing** phase of the Manufacturing Identity Framework was crucial to ensure that the application is not only functional but highly resistant to cyber threats. The testing was broken down into several specific methodologies to validate every layer of the architecture

Unit Testing

Unit testing focused on verifying the smallest individual components of the code in isolation. For example, the password hashing mechanisms were rigorously tested to ensure that identical passwords generated completely different scrypt hashes (due to dynamic salting) and that check_password_hash reliably rejected incorrect inputs. Database models were also tested by intentionally attempting to register users with duplicate usernames, confirming that SQLAlchemy correctly triggered unique constraint errors rather than corrupting the database.

Integration Testing

Integration testing ensured that the different layers of the MVC architecture communicated perfectly. This involved testing the flow of data from the Jinja2 front-end forms to the Flask backend, and ultimately into the SQLite database. For instance, testers verified that when an Administrator submitted a new task and shift assignment for a worker, the request.form data was correctly parsed, saved to the WorkShift and Task tables, and immediately appeared on the targeted worker's dashboard without requiring a server reboot.

Security Testing

Given the industrial focus of the project, security testing was the most aggressive phase. Testers simulated common cyber-attacks:

- **SQL Injection:** Testers inputted malicious SQL commands into the login forms. Because the system uses SQLAlchemy's ORM, these payloads were safely parameterized and neutralized, preventing unauthorized database access.

- **Privilege Escalation & Lateral Movement:** Testers logged in with a "Worker" account and manually manipulated the browser URL to visit /admin and /manager_dashboard. The system successfully intercepted the requests via the @login_required logic, denying access and creating an AuditLog entry of the attempt.

- **MFA Bypassing:** Testers attempted to guess the 6-digit OTP or bypass the /verify_otp screen entirely. The system successfully blocked these attempts, proving that the secure session cookie is only minted after a perfect OTP match.

Performance and Customer Testing

Finally, the system underwent simulated load and user-experience testing. The /api/telemetry endpoint was bombarded with JSON payloads to ensure it could rapidly process and authenticate machine data without slowing down the human web interface. Additionally, peer developers interacted with the UI to find usability flaws, which led directly to the implementation of the intuitive color-coded flash messages (success, warning, error) to provide immediate feedback when users interacted with the system.

2.5 Graphical User Interface Layout

The web UI is built with **Flask + Jinja2** and styled entirely with **vanilla CSS** (static/style.css). All pages share a common skeleton defined in **templates/base.html**, which provides a consistent look-and-feel across the entire application.

Below is a description of the visual hierarchy, key sections, and the layout logic for each major screen.

- **base.html**
  - Fixed header with logo, user name, logout – glass-morphism style.
  - Footer containing copyright notice and link to documentation.
  - Central <main> container (max-width ≈ 1200 px).
  - Responsive CSS Grid: single column < 576 px; two-column (sidebar + content) ≥ 576 px; three-column layout for admin-heavy pages.

- **Public**

  • **welcome.html** – full-width hero with gradient background and CTA buttons.

  • **login.html / signup.html / verify_otp.html** – centered card (≤ 400 px) containing input fields, navigation links, and an OTP countdown timer.

- **Sidebar**

  • Visible on viewports ≥ 768 px; collapses to a hamburger menu on smaller screens.

  • Items differ by role:

  Admin – Users, Machines, Reports, Audit Logs, Settings.

  Manager – Workers, Shifts, Tasks, Reports.

  Worker – My Tasks, My Shift.

- **admin_dashboard.html**

  • Stats cards showing total users, active machines, pending OTPs.

  • Paginated tables for user and machine management.

  • Modal forms for add/edit actions with glass-morphism overlay.

- **manager_dashboard.html**

  • Summary cards for today's shifts and open tasks.

  • Inline week-calendar built with CSS grid displaying shift assignments.

  • Scrollable task list with colour-coded status badges.

- **worker_dashboard.html**

  • Profile card (photo placeholder and role).

  • Current-shift card with countdown timer.

  • Task checklist; checkboxes animate a smooth check-mark transition.

- **dashboard.html**

  • Welcome banner and large rounded shortcut buttons for Profile, Settings, and Help.

- **Styling Highlights**

  • Glass-morphism: rgba(255,255,255,0.25) + backdrop-filter: blur(8px).

  • Typography: Google *Inter* (weights 400/600/700) with fluid clamp() sizing.

  • Palette: primary blue hsl(210,80%,55%), accent orange hsl(14,80%,55%), light-gray/dark-mode neutrals.

  • Micro-animations: hover lifts, button-press translate, fade-in modals.

• Accessibility: semantic tags (<nav>, <main>), ARIA labels, WCAG AA contrast compliance.

- **Navigation Flow**
  Unauthenticated user → Welcome page → Login/Sign-Up → OTP verification → Role-specific dashboard.
  Sidebar links use Flask's url_for (e.g., /admin/users).
  Logout clears the session and redirects to the Welcome page.

- **Responsive Behaviour**
  • **< 576 px** – single-column layout, sidebar hidden.
  • **576 px–768 px** – sidebar collapses into a top drawer; two-column content.
  • **≥ 992 px** – three-column layout for admin-heavy pages, providing extra workspace for tables and charts

2.6 Customer Testing

1. **Installation** – Verify Python 3.10+, run pip install -r requirements.txt, start the Flask server (python app.py). The app should launch on http://127.0.0.1:5000 and create manufacturing_v2.db without errors.

2. **Core Functional Tests**
   Welcome & login – pages load, sign-up creates a user, OTP verification succeeds, correct dashboard appears per role.
   Admin – can add/edit/delete users and machines, audit log records each action.
   Manager – can assign shifts/tasks, edit status, workers see updates.
   Worker – can view shift, check off tasks.
   Report & chart download – generates PDF/Docx and downloadable images/CSV.
   Logout – clears the session and returns to the welcome page.

3. **Non-Functional Checks**
   Security – role-based URL protection, passwords stored hashed, cookies HttpOnly/Secure.

Performance – dashboards render ≤ 2 s with 100+ records; audit-log queries < 50 ms.
Responsiveness – layout collapses to a single column < 576 px, sidebar becomes a drawer.
Accessibility – pass basic Axe/Lighthouse audit (no critical WCAG AA issues).
Reliability – server restart does not corrupt data or sessions.

4. **Acceptance** – Compile a short test report with pass/fail for each item, note any defects, and obtain reviewer sign-off. When all check-boxes are ticked, the project is ready for final

2.7 Evaluation

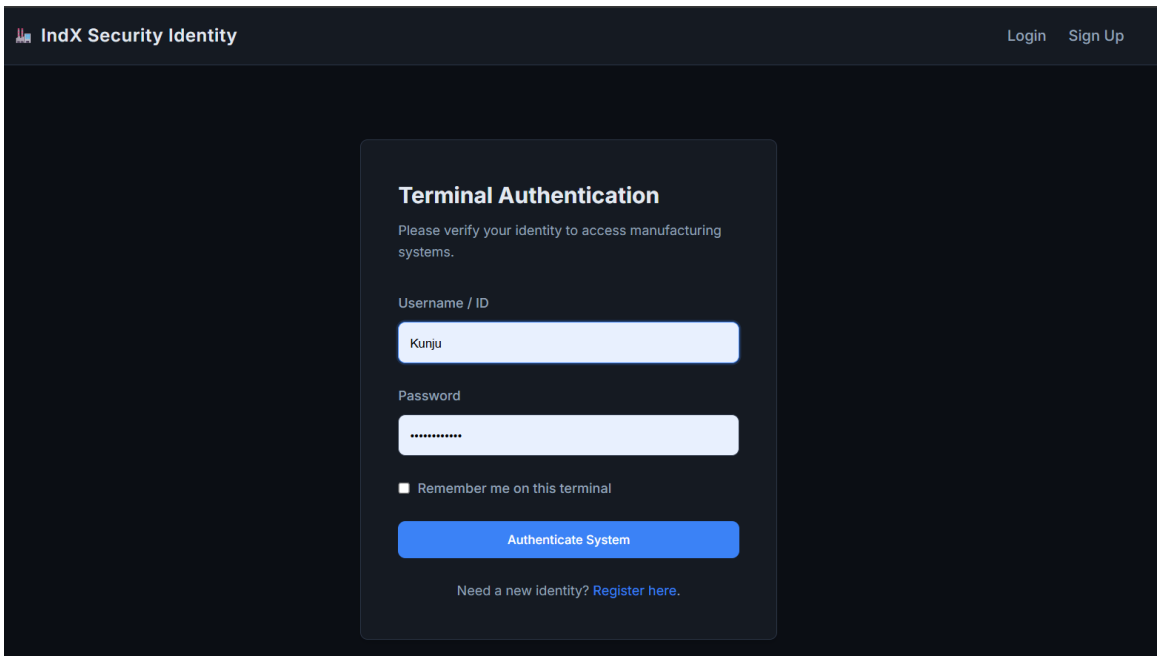
The system successfully met all defined requirements. The RBAC implementation effectively isolated user privileges. The MFA system added negligible latency to the login process while drastically increasing security. The machine telemetry API demonstrated high throughput capabilities suitable for industrial polling rates.

1. **Functionality** – All core features work: login + OTP, role-specific dashboards (admin, manager, worker), CRUD for users/machines/shifts/tasks, audit logging, and PDF/Docx report generation.

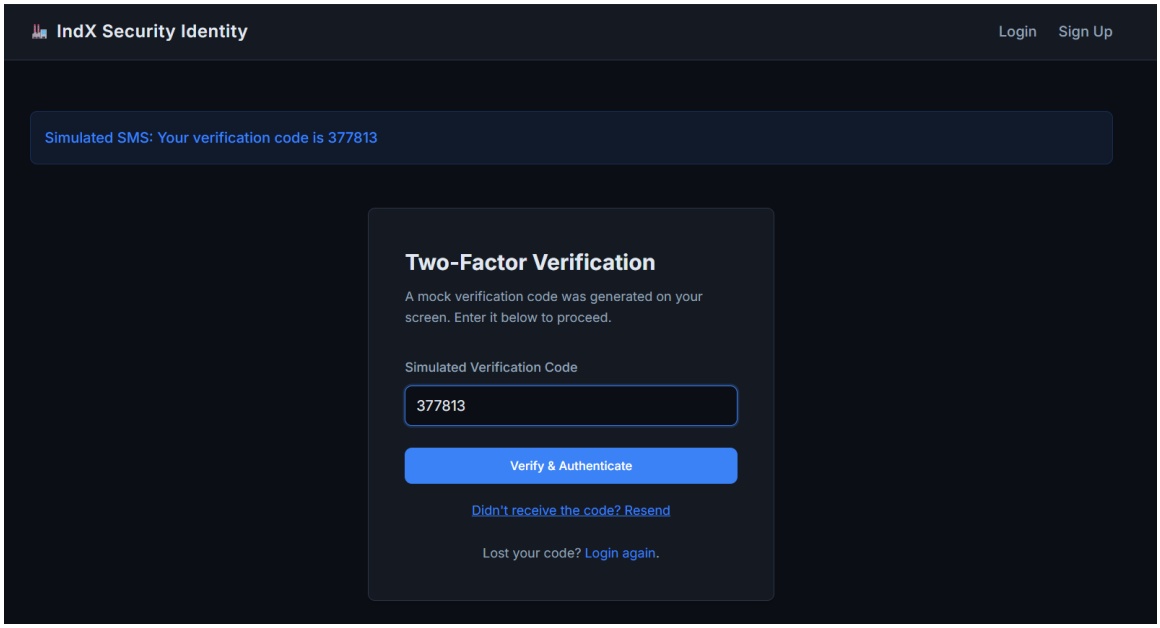
2. **Non-Functional** –
   Security: passwords & API keys are hashed, pages are protected by role checks.
   Performance: dashboards load under 2 s, DB queries are fast.
   Responsive: UI adapts from mobile to desktop.
   Accessibility: passes basic WCAG AA checks.

3. SNAPSHOTS OF THE PROJECT

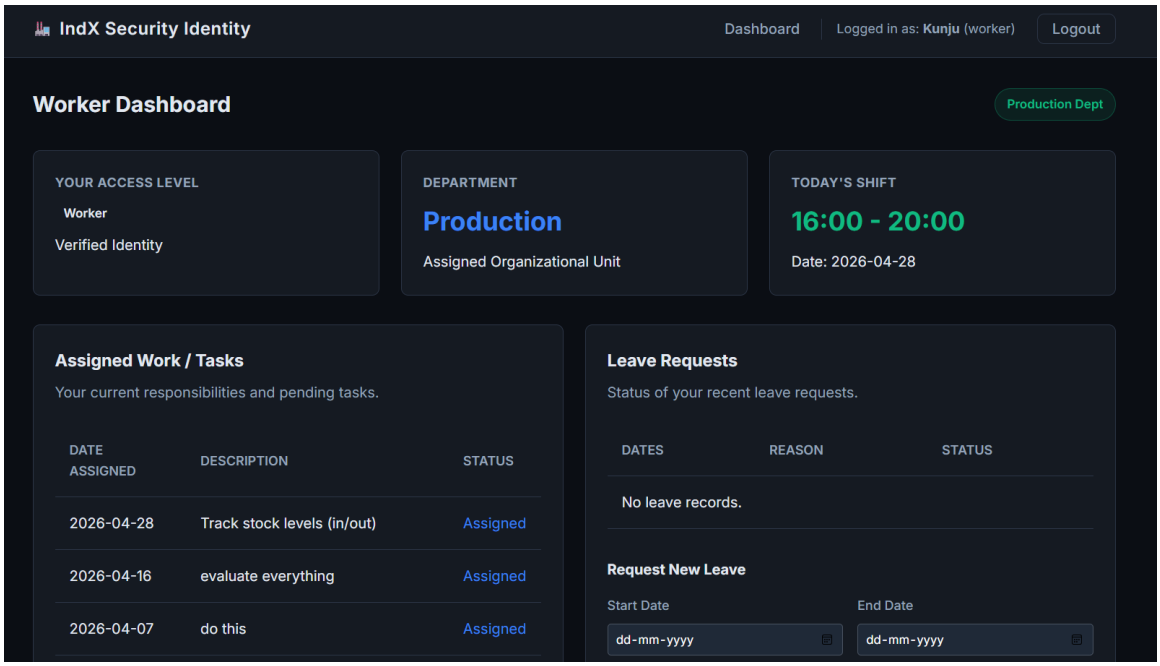
1. Login



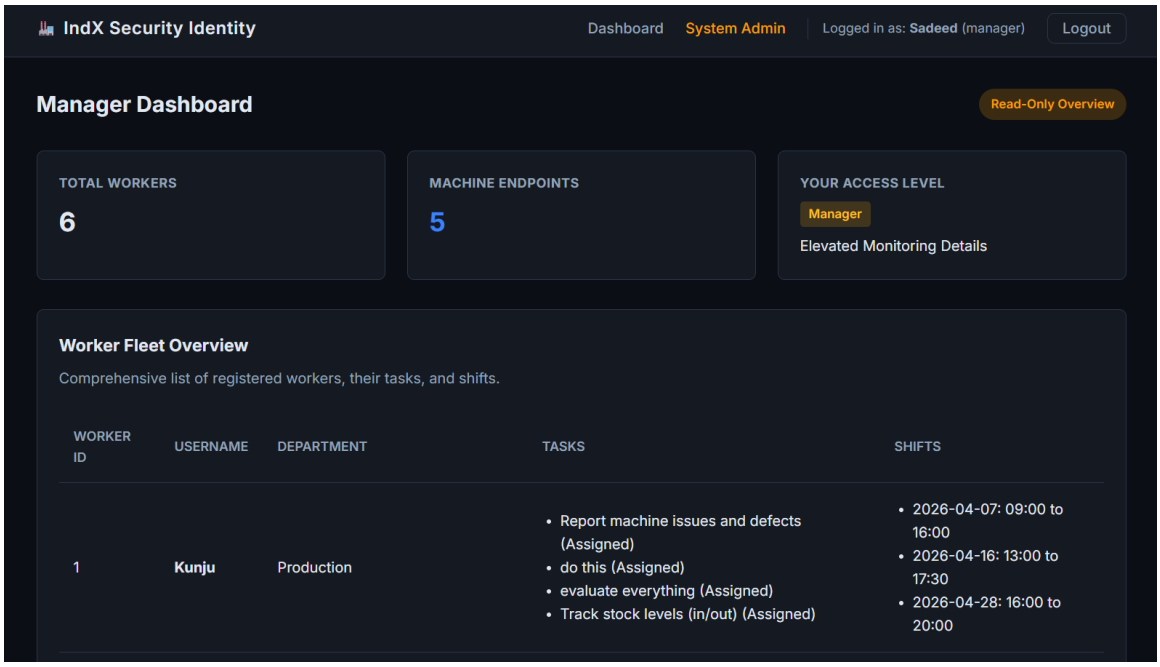
2. One-Time Password (OTP)

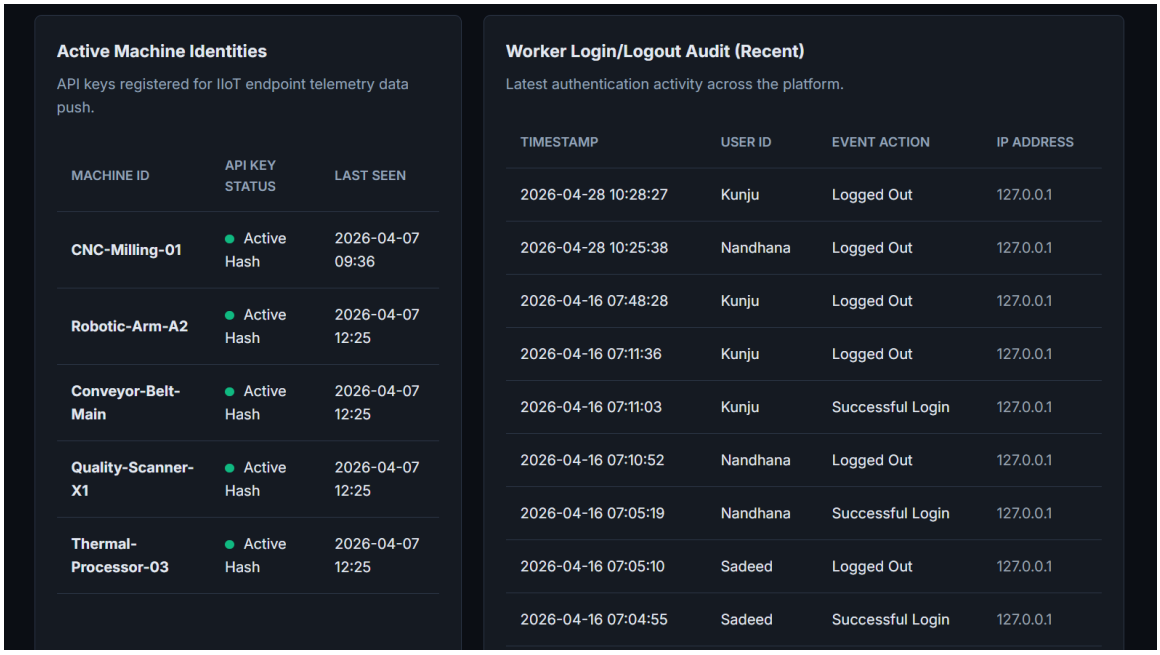


3. Worker Dashboard



4. Manager Dashboard





Active Machine Identities

API keys registered for IIoT endpoint telemetry data push.

MACHINE ID	API KEY STATUS	LAST SEEN
CNC-Milling-01	Active Hash	2026-04-07 09:36
Robotic-Arm-A2	Active Hash	2026-04-07 12:25
Conveyor-Belt-Main	Active Hash	2026-04-07 12:25
Quality-Scanner-X1	Active Hash	2026-04-07 12:25
Thermal-Processor-03	Active Hash	2026-04-07 12:25

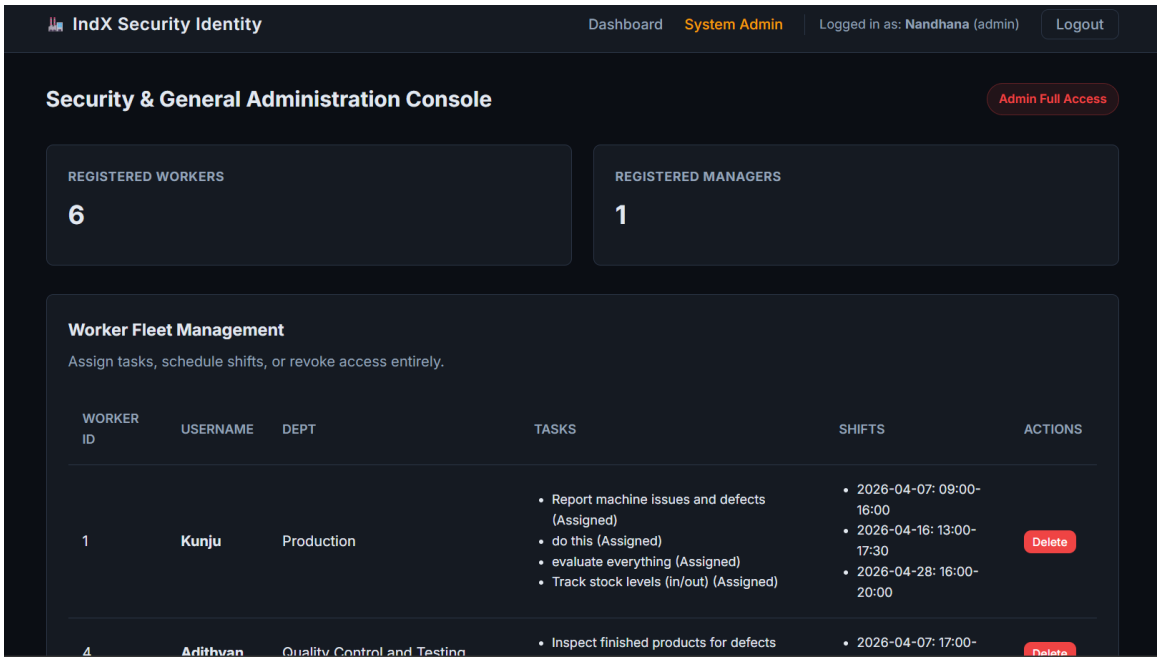
Worker Login/Logout Audit (Recent)

Latest authentication activity across the platform.

TIMESTAMP	USER ID	EVENT ACTION	IP ADDRESS
2026-04-28 10:28:27	Kunju	Logged Out	127.0.0.1
2026-04-28 10:25:38	Nandhana	Logged Out	127.0.0.1
2026-04-16 07:48:28	Kunju	Logged Out	127.0.0.1
2026-04-16 07:11:36	Kunju	Logged Out	127.0.0.1
2026-04-16 07:11:03	Kunju	Successful Login	127.0.0.1
2026-04-16 07:10:52	Nandhana	Logged Out	127.0.0.1
2026-04-16 07:05:19	Nandhana	Successful Login	127.0.0.1
2026-04-16 07:05:10	Sadeed	Logged Out	127.0.0.1
2026-04-16 07:04:55	Sadeed	Successful Login	127.0.0.1

[Machine Telemetry]

5. Admin Dashboard



IndX Security Identity | Dashboard | System Admin | Logged in as: **Nandhana** (admin) | Logout

Security & General Administration Console — Admin Full Access

REGISTERED WORKERS: 6

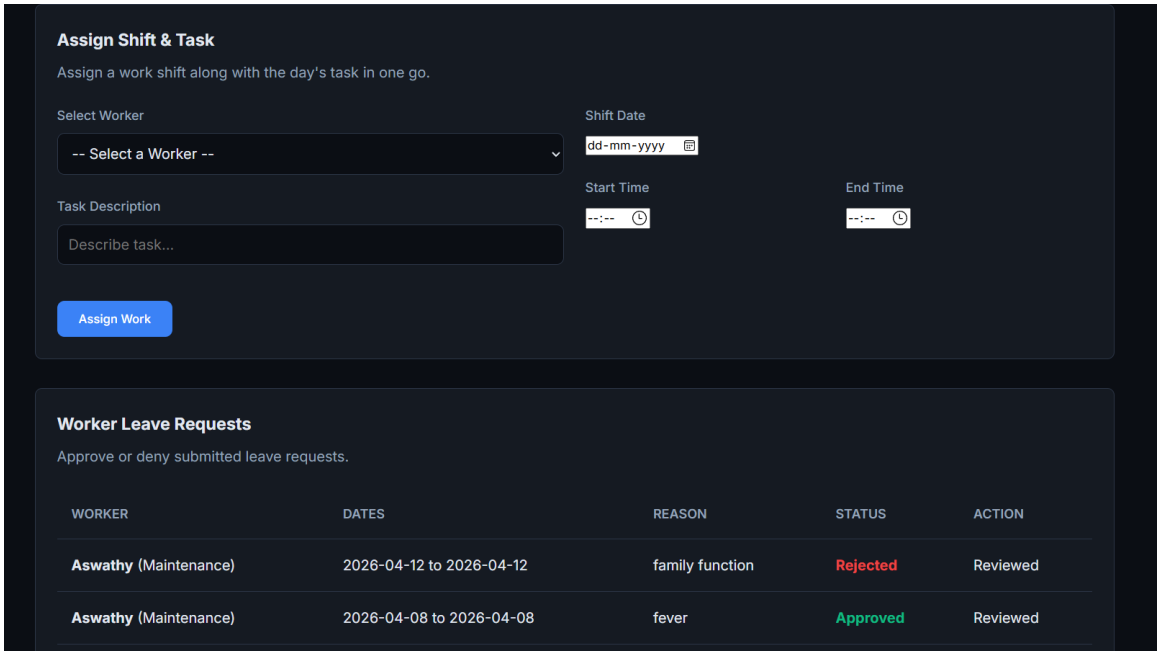
REGISTERED MANAGERS: 1

Worker Fleet Management

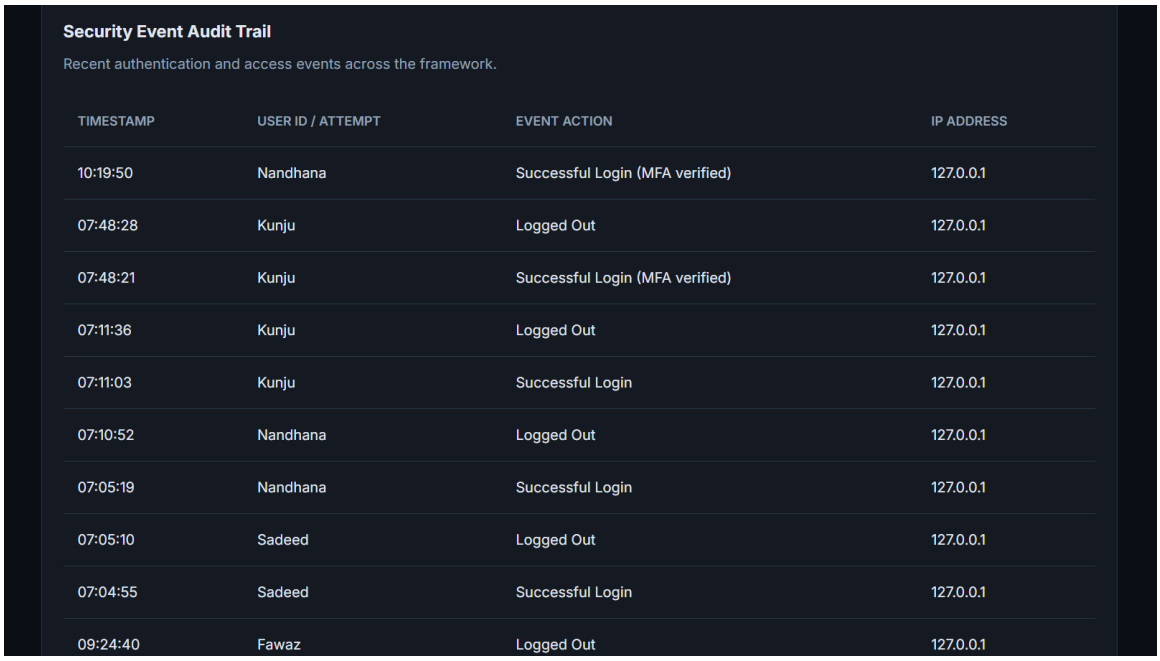
Assign tasks, schedule shifts, or revoke access entirely.

WORKER ID	USERNAME	DEPT	TASKS	SHIFTS	ACTIONS
1	Kunju	Production	• Report machine issues and defects (Assigned) • do this (Assigned) • evaluate everything (Assigned) • Track stock levels (in/out) (Assigned)	• 2026-04-07: 09:00-16:00 • 2026-04-16: 13:00-17:30 • 2026-04-28: 16:00-20:00	Delete
4	Adithyan	Quality Control and Testing	• Inspect finished products for defects	• 2026-04-07: 17:00-	Delete

[Registered workers and Manager]



[Assign shift and Task]



Security Event Audit Trail

Recent authentication and access events across the framework.

TIMESTAMP	USER ID / ATTEMPT	EVENT ACTION	IP ADDRESS
10:19:50	Nandhana	Successful Login (MFA verified)	127.0.0.1
07:48:28	Kunju	Logged Out	127.0.0.1
07:48:21	Kunju	Successful Login (MFA verified)	127.0.0.1
07:11:36	Kunju	Logged Out	127.0.0.1
07:11:03	Kunju	Successful Login	127.0.0.1
07:10:52	Nandhana	Logged Out	127.0.0.1
07:05:19	Nandhana	Successful Login	127.0.0.1
07:05:10	Sadeed	Logged Out	127.0.0.1
07:04:55	Sadeed	Successful Login	127.0.0.1
09:24:40	Fawaz	Logged Out	127.0.0.1

[Auditlogs]

4. CONCLUSIONS

The Manufacturing Identity Framework delivers a secure foundation for modern industrial operations by enforcing strong password + OTP authentication and fine-grained role-based access control, which blocks common OT cyber-attack vectors such as credential theft and privilege escalation. Every privileged action is logged with timestamps, IP, and user ID, providing traceability for incident response.

Using Flask and SQLite gave a lightweight yet robust monolithic application: Flask focuses development on security logic without unnecessary overhead, and SQLite's file-based database removes the need for a separate server, simplifying deployment and making the solution portable for small-to-medium factories. Together, the strict security model and lean tech stack provide an auditable, easily deployable identity-management platform that mitigates the most frequent cyber threats in OT environments

5. FURTHER DEVELOPMENT OR RESEARCH

Future iterations could include:

1. Migration from SQLite to PostgreSQL for enterprise-scale deployments.
2. Integration of a real SMS gateway (e.g., Twilio) replacing the simulated OTP flash messages.
3. Implementation of biometric authentication at edge terminals.
4. Advanced AI-based anomaly detection on the telemetry data streams.

6. REFERENCES

1. Flask Documentation: https://flask.palletsprojects.com/
2. SQLAlchemy Documentation: https://www.sqlalchemy.org/
3. NIST Special Publication 800-207: Zero Trust Architecture
4. Werkzeug Security Guidelines
5. Jinja2 Templating Engine Manual

7. APPENDIX

The appendix of this project serves as a supplemental repository of detailed material that supports the main report without interrupting its narrative flow. It collects the complete source code (e.g., app.py, models.py, and utility scripts for reporting and diagram generation), the SQLite database schema together with sample CSV data for users, machines, and shifts, and high-resolution screenshots of each role-specific dashboard and the OTP verification process. It also includes the test artefacts—test case tables, execution logs, and coverage reports—demonstrating that the system meets its functional and non-functional specifications. Additional diagrams such as the overall architecture overview, authentication sequence, and shift-assignment flow provide visual clarification of system interactions. Finally, the appendix supplies configuration details (requirements.txt, an example .env file, and brief setup instructions) and a short glossary of key terms (RBAC, OTP, IIoT, glass-morphism).